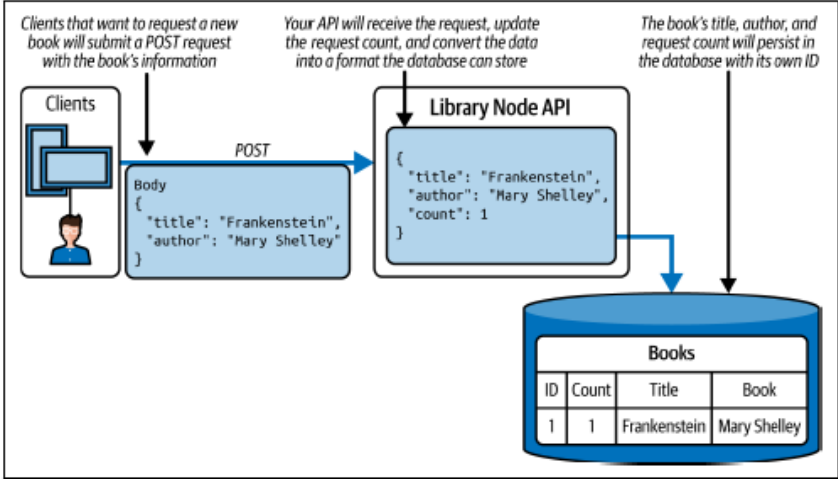


	ITER, SIKSHA 'O' ANUSANDHAN (Deemed to be University)		Minor Project
Branch	CSIT	Programme	B.Tech
Course Name	Full-Stack Web Development with MERN	Semester	6th
Course Code	CSE 3839	Academic Year	2025-2026
Minor Project			
Learning Level (LL)	L1: Remembering	L3: Applying	L5: Evaluating
	L2: Understanding	L4: Analysing	L6: Creating
Q's	Questions		COs
1	Design and implement a full-stack Library Management System using the MERN stack that allows users to perform CRUD operations on books. The system should include a React frontend interface for users to interact with the API and support image upload/display for books.  Develop a responsive UI that includes: Add Book Form Title Author Upload Book Image Book List Display Show: Book Image Title Author Count Display all books dynamically User Actions Add new book View all books		CO1,CO2, CO3,CO4
LL		L3,L4,L6	

	Implement REST APIs: POST /api/books → Add book or update count GET /api/books → Fetch all books PUT /api/books/:id → Update book DELETE /api/books/:id → Delete book		
2	In this project, apply MongoDB database to store and manage application data efficiently. Two main collections are to be created: a) Books Collection This collection stores the details of all books available in the library. Each document in this collection contains: Title: The name of the book Author: The writer of the book Count: Number of times the book has been requested This collection helps in managing book records and tracking their popularity based on request count. b) Requests Collection This collection stores details of book requests made by users. Each document includes: Book Title: Name of the requested book Requested By: Name or ID of the user Date: Request date This allows tracking of user activity and request history in the application. c) Implement logic such that: When a new book request is made: If the book already exists → Increment its count field If the book does not exist → Create a new book with count = 1 This avoids duplicate entries and ensures efficient data storage. d) Optimize Page Titles: Provide unique and descriptive titles for each page (Home, Books, Add Book Form). Improve URL Structure: Use clean and meaningful URLs (e.g., /books, /add-book). Image Optimization: Add alt attributes to book images and compress them for faster loading.	CO5	L3,L4
3	Design and implement data aggregation features using MongoDB to analyze book data stored in the database, along with interactive frontend	CO6	L3,L4

	functionality for users. a) Display all book records retrieved from the database. Show details such as Title, Author, and Count b) Implement filtering functionality: Filter books based on request count (e.g., count < 5) Filter books within a range (e.g., count between 5–10) c) Update the UI dynamically based on selected filter criteria. Ensure filtered results are displayed instantly without reloading the page d) Add a “Request Book” button to each book card. e) On clicking the button: Display a confirmation message (e.g., “Book requested successfully”) Send a request to the backend API f) Implement backend aggregation logic: If the book already exists → Increment its count If the book does not exist → Create a new entry with count = 1		
--	---	--	--

Note:

1. Minor Project carries a weightage of 20 **marks out of 100**
2. All course outcomes were covered.

Course Outcomes	CO1	Understand and configure a modern full-stack development environment using Node.js, MongoDB, Docker, and supporting tools.
	CO2	Design and implement RESTful backend services using Express, MongoDB (Mongoose), and unit testing frameworks.
	CO3	Develop interactive frontend applications using React and efficiently integrate them with backend APIs using modern data-fetching techniques.
	CO4	Apply DevOps practices to containerize, test, and deploy full-stack applications using Docker and CI/CD pipelines.
	CO5	Implement authentication, performance optimization, Search Engine Optimization techniques, and automated testing for scalable and secure web applications.
	CO6	Build and integrate advanced full-stack architectures using data aggregation, visualization, and GraphQL APIs with modern frontend clients.